
Cielo y fondos

El realismo de una escena 3D suele mejorar al generar un efecto realista en el horizonte lejano. Al observar más allá de los edificios y árboles cercanos, estamos acostumbrados a ver objetos distantes como nubes, montañas o el sol (o por la noche, la luna y las estrellas). Sin embargo, añadir estos objetos a la escena como modelos individuales puede afectar negativamente al rendimiento. Un cielo virtual (skybox o skydome) ofrece una forma sencilla y eficaz de generar un horizonte convincente.

Cielos virtuales

El concepto de *cielo virtual* es ingenioso y simple:

1. Crear un objeto cúbico.
2. Aplicar una textura al cubo que represente el entorno deseado.
3. Posicionar el cubo de forma que rodee la cámara.

Ya conocemos estos pasos. Sin embargo, hay algunos detalles adicionales.

- *¿Cómo creamos la textura del horizonte?*
Un cubo tiene seis caras, y todas ellas necesitan una textura. Una opción es usar seis imágenes y seis unidades de textura. Otra opción común (y eficiente) es usar una sola imagen que contenga las texturas de las seis caras, como se muestra en la figura 1.

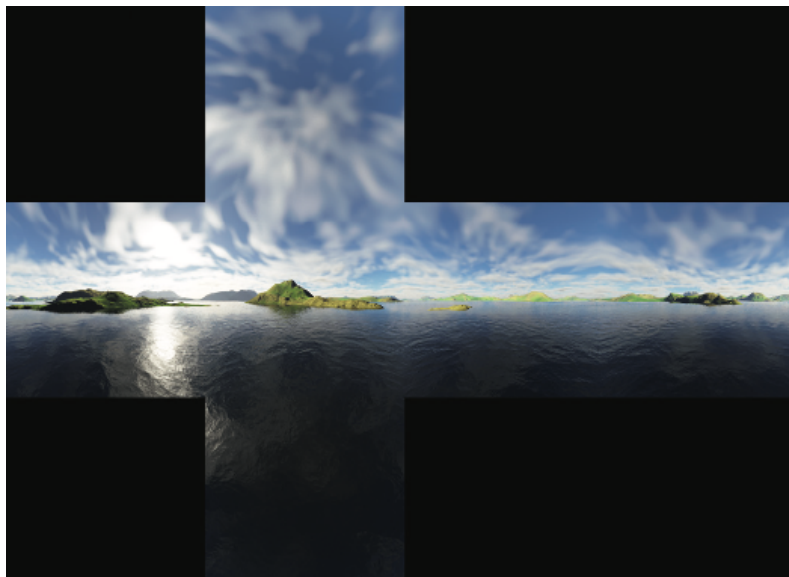


Figura 1: Mapa de textura de cubo para cielo virtual (6 caras).

Una imagen que permite texturizar las seis caras de un cubo con una sola unidad de textura se denomina *mapa de textura de cubo*. Las seis partes del mapa corresponden a la parte superior, inferior, frontal, posterior y los dos laterales del cubo. Al envolver el cubo con esta textura, simula un horizonte para una cámara situada en su interior, como se muestra en la figura 2.

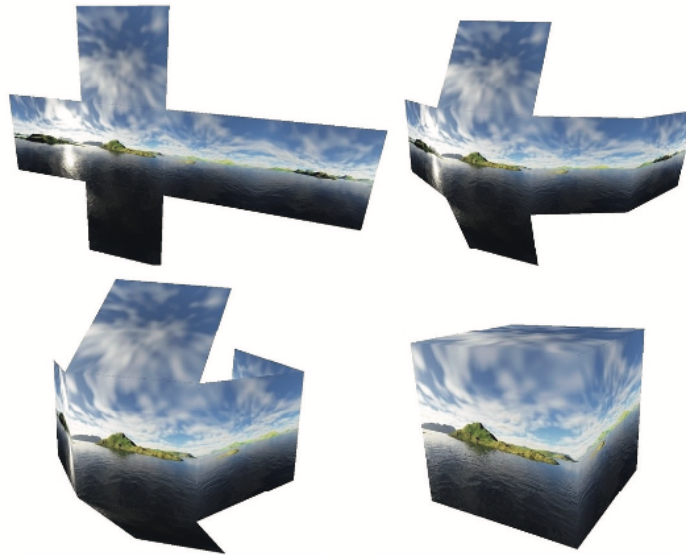


Figura 2: Mapa de textura de cubo envolviendo la cámara.

Para texturizar el cubo con un mapa de textura de cubo, se deben especificar las coordenadas de textura adecuadas. La figura 3 muestra la distribución de las coordenadas de textura asignadas a cada vértice del cubo.

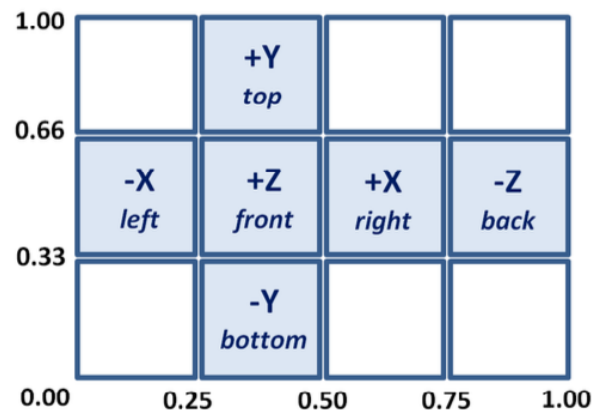


Figura 3: Coordenadas de textura del mapa de cubo.

- ¿Cómo conseguimos que el cielo virtual parezca lejano?
Otro factor importante es asegurar que la textura parezca un horizonte lejano. Inicialmente, uno podría pensar que esto requiere un cielo virtual muy grande. Sin

embargo, esto no es recomendable, ya que distorsionaría la textura. En cambio, es posible hacer que el fondo de cielo parezca muy grande (y, por lo tanto, distante) mediante el siguiente truco en dos pasos:

- Deshabilitar la prueba de profundidad y renderizar primero el fondo de cielo (habilitando posteriormente la prueba de profundidad al renderizar los demás objetos de la escena).
- Mover el fondo de cielo junto con la cámara (si esta se mueve).

Al dibujar primero el fondo de cielo con la prueba de profundidad deshabilitada, el búfer de profundidad se llenará completamente con valores de 1.0 (es decir, con la máxima distancia). De esta forma, todos los demás objetos de la escena se renderizarán correctamente; es decir, ningún otro objeto quedará oculto por el fondo de cielo. Esto hace que las paredes del fondo de cielo parezcan más distantes que cualquier otro objeto, independientemente de su tamaño real. El cubo que representa el fondo de cielo puede ser muy pequeño, siempre y cuando se mueva junto con la cámara. La figura 4 muestra una escena simple (un toro con textura de ladrillo) vista desde dentro del fondo de cielo.

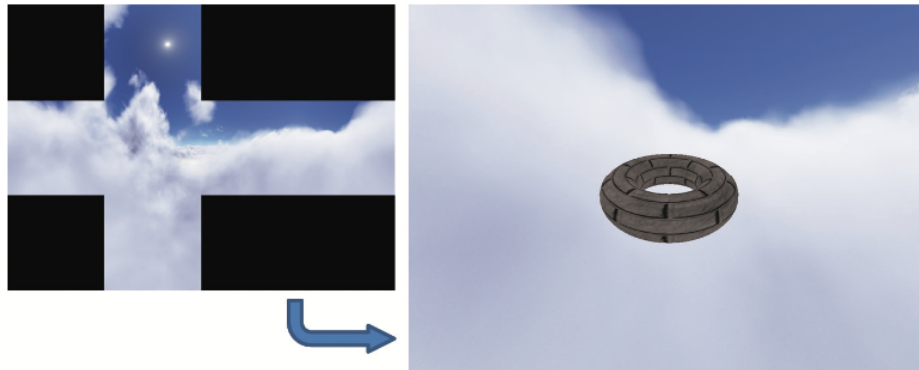


Figura 4: Visualización de una escena desde el interior de un cubo de cielo.

Es útil examinar detenidamente la figura 4 en relación con las figuras 2 y 3. Obsérvese que la parte del cubo de cielo visible en la escena corresponde a la sección derecha del mapa de textura cúbica. Esto se debe a que la cámara está en la orientación predeterminada, apuntando en dirección negativa del eje Z, por lo que está viendo la parte posterior del cubo (como se indica en la figura 3).

Además, esta parte posterior del mapa de textura aparece invertida horizontalmente al renderizarse, ya que se visualiza desde el interior del cubo. Por ejemplo, la parte posterior (-Z) del mapa de textura se ve plegada alrededor de la cámara y, por lo tanto, aparece invertida, como se muestra en la figura 2.

- **¿Cómo se crea un mapa de textura cúbica?**

Crear una imagen de mapa de textura cúbica a partir de imágenes o fotografías requiere cuidado para **evitar las líneas de unión entre las caras del cubo** y para lograr una perspectiva correcta, de modo que el cubo de cielo parezca realista y sin distorsiones.

Existen varias herramientas para ello: **Terragen, Autodesk 3ds Max, Blender y Adobe Photoshop** ofrecen herramientas para crear o trabajar con mapas de textura cúbicos. También hay muchas páginas web que ofrecen diversos mapas de textura cúbicos predefinidos, algunos gratuitos y otros de pago.

Domos de cielo

Otra forma de crear un efecto de horizonte es mediante un **domo de cielo**. La idea básica es la misma que con el cubo de cielo, pero en lugar de un cubo, se usa una esfera (o media esfera) con textura. Al igual que con el cubo de cielo, **primero se renderiza el domo** (con la prueba de profundidad desactivada) **y la cámara se coloca en el centro**. (La textura del domo de la figura 5 se creó con Terragen¹.)

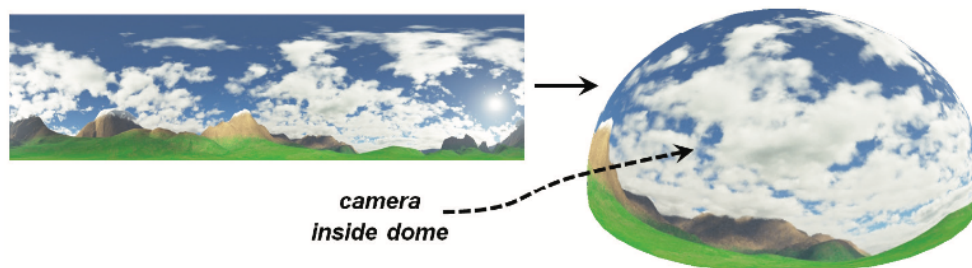


Figura 5: Domo de cielo con la cámara en su interior.

Los domos de cielo tienen algunas ventajas respecto a los cubos de cielo. Por ejemplo, **son menos propensos a distorsiones y líneas de unión** (aunque la distorsión esférica en los polos debe tenerse en cuenta en la imagen de textura). **Una desventaja** de la bóveda celeste es que **una esfera o cúpula es un modelo más complejo que un cubo**, con muchos **más vértices** y un número de vértices potencialmente variable según la precisión deseada.

Al usar una **bóveda celeste** para representar una escena exterior, **suele combinarse con un plano de suelo o algún tipo de terreno**. Para representar una escena espacial, como un cielo estrellado, suele ser más práctico usar una esfera, como se muestra en la figura 6 (se ha añadido una línea discontinua para facilitar la visualización de la esfera).

¹ Terragen, Planetside Software: <https://planetside.co.uk/>

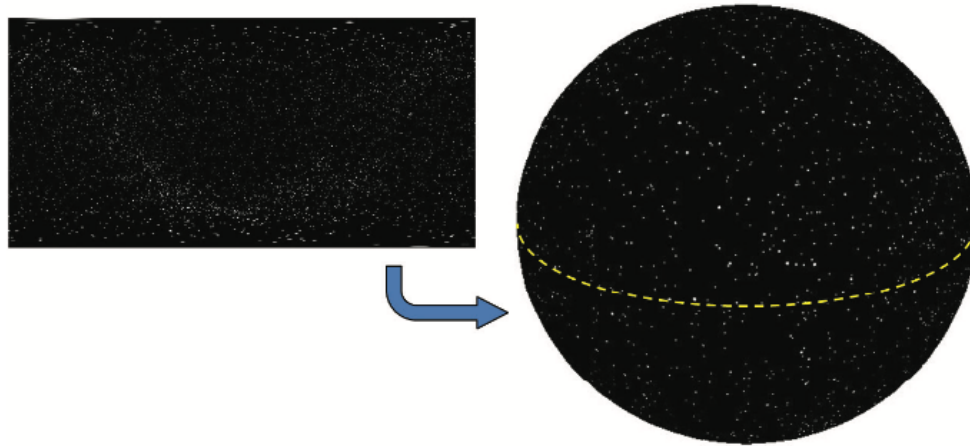


Figura 6: Bóveda celeste de estrellas usando una esfera (cielo estrellado de P. Bourke²).

Implementación de un cubo de cielo

A pesar de las ventajas de la bóveda celeste, los cubos de cielo siguen siendo más comunes. Además, tienen mejor soporte en OpenGL, lo cual es ventajoso para el mapeo ambiental (tema que se aborda más adelante en este documento). Por estas razones, nos centraremos en la implementación de un cubo de cielo.

Existen dos métodos para implementar un cubo de cielo: crear uno simple desde cero y usar las funciones de mapeo de cubos en OpenGL. Cada método tiene sus ventajas, por lo que los abordaremos ambos.

Crear un cubo de cielo desde cero

Ya hemos cubierto casi todo lo necesario para crear un cubo de cielo simple. En el documento 4 del curso se presentó un modelo de cubo; podemos asignar las coordenadas de textura como se muestra en la figura 3 de este documento. Vimos cómo cargar texturas usando la biblioteca SOIL2 y cómo posicionar objetos en el espacio 3D. Veremos cómo habilitar y deshabilitar fácilmente la prueba de profundidad (con una sola línea de código).

El Programa 9.1 muestra la organización del código para nuestro cubo de cielo simple, con una escena que consiste en un simple toro con textura. Se resaltan las asignaciones de coordenadas de textura y las llamadas para habilitar/deshabilitar la prueba de profundidad.

Programa 9.1: Cubo de cielo simple
(Revisar el código adjunto)

² <https://paulbourke.net/miscellaneous/astronomy/>

La salida del Programa 9.1 se muestra en la Figura 7, para dos texturas de mapeo de cubo diferentes.

Como se mencionó anteriormente, los cubos de cielo son susceptibles a distorsiones de imagen y a líneas de borde (*seams*). Las líneas de borde son líneas visibles a veces donde se unen dos imágenes de textura, como en los bordes del cubo. La figura 8 muestra un ejemplo de una línea de borde en la parte superior de la imagen, un artefacto del Programa 9.1. Para evitar las imperfecciones en las uniones, es necesario crear cuidadosamente la imagen del mapa de cubo y asignar las coordenadas de textura con precisión. Existen herramientas para minimizar estas imperfecciones en los bordes de la imagen (sin embargo, este tema no se aborda en este curso).

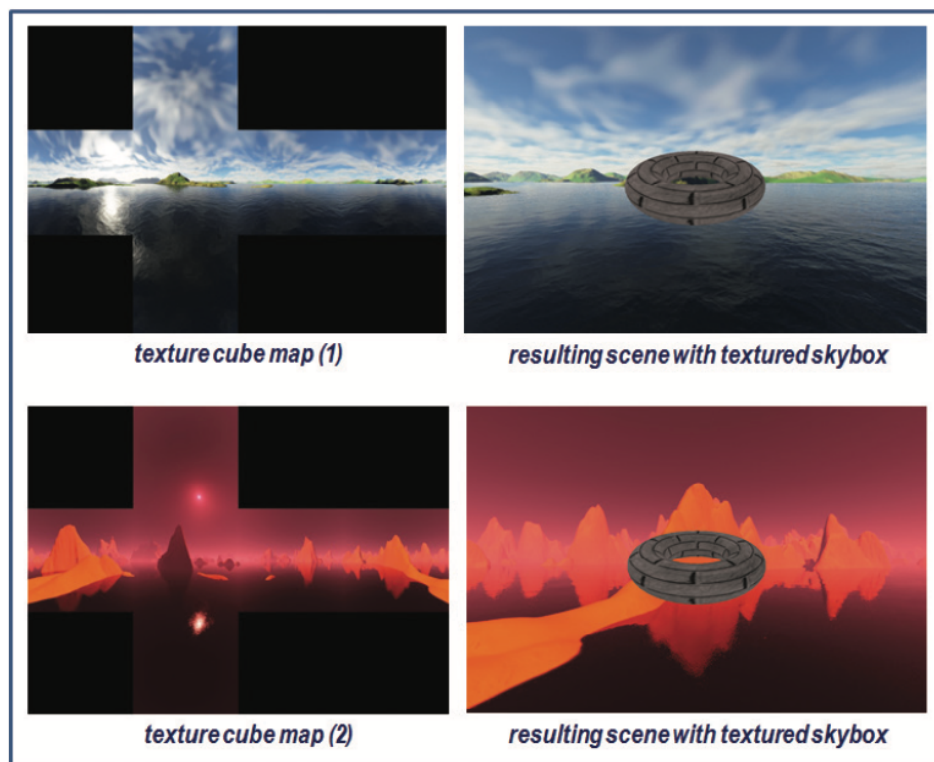


Figura 7: Resultados de un cielo virtual sencillo.

Uso de mapas de cubo en OpenGL

Otra forma de crear un cielo es mediante el uso de un mapa de textura de cubo en OpenGL. Los mapas de cubo de OpenGL son algo más complejos que el método sencillo que vimos en la sección anterior. Sin embargo, tienen ventajas, como la reducción de las imperfecciones en las uniones y el soporte para el mapeo ambiental.

Los mapas de textura de cubo de OpenGL son similares a las texturas 3D que estudiaremos más adelante, ya que se accede a ellos mediante tres coordenadas de textura (a menudo etiquetadas como (s, t, r)), en lugar de dos como hemos hecho hasta ahora. Otra característica particular es que las imágenes en estos mapas están orientadas con la coordenada de textura $(0,0,0)$ en la esquina superior izquierda (en lugar de la habitual inferior izquierda); esto suele generar confusión.

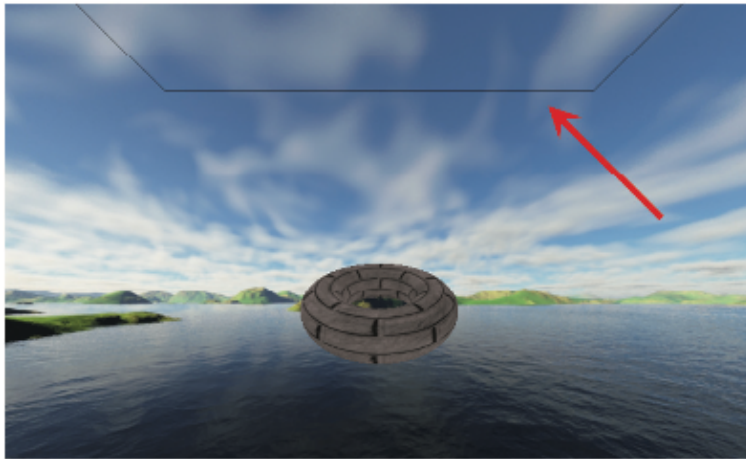


Figura 8: Imperfección en la unión de un cielo.

Mientras que el método mostrado en el Programa 9.1 lee una única imagen para texturizar el mapa de cubo, la función `loadCubeMap()` del Programa 9.2 lee seis archivos de imagen separados, uno para cada cara del cubo. Como vimos en el documento 5, existen muchas maneras de cargar imágenes de textura; elegimos usar la biblioteca SOIL2. Aquí, SOIL2 también resulta muy útil para crear e inicializar un mapa de cubo en OpenGL.

Se identifican los archivos a cargar y luego se llama a `SOIL_load_OGL_cubemap()`. Los parámetros incluyen los seis archivos de imagen y otros parámetros similares a los de `SOIL_load_OGL_texture()` que vimos en el documento 5. En el caso de los mapas de cubo de OpenGL, no es necesario invertir las texturas verticalmente, ya que OpenGL lo hace automáticamente. Nota que hemos colocado `loadCubeMap()` en el archivo “Utils.cpp”.

La función `init()` ahora incluye una llamada a `GL_TEXTURE_CUBE_MAP_SEAMLESS`, que indica a OpenGL que intente mezclar los bordes adyacentes del cubo para reducir o eliminar las imperfecciones.

En `display()`, los vértices del cubo se envían al pipeline como antes, pero esta vez no es necesario enviar las coordenadas de textura. Como veremos, esto se debe a que un mapa de textura de cubo de OpenGL suele usar las posiciones de los vértices del cubo como coordenadas de textura. Tras desactivar la prueba de profundidad, se dibuja el cubo. A continuación, se vuelve a activar la prueba de profundidad para el resto de la escena.

El mapa de textura cúbica OpenGL finalizado se referencia mediante un identificador entero. Al igual que en el caso del mapeo de sombras, se pueden reducir los artefactos en los bordes configurando el modo de envoltura de textura en "clamp to edge". Esto ayuda a reducir aún más las imperfecciones. Ten en cuenta que esta configuración se aplica a las tres coordenadas de textura: s, t y r.

La textura se accede en el sombreador de fragmentos con un tipo especial de muestreador llamado samplerCube. En un mapa de textura cúbica, el valor devuelto por el muestreador es el texel "visto" desde el origen, según el vector de dirección (s, t, r). Por lo tanto, normalmente podemos usar las posiciones de vértice interpoladas como coordenadas de textura. En el sombreador de vértice, asignamos las posiciones de los vértices del cubo al atributo de coordenadas de textura para que se interpolen al llegar al sombreador de fragmentos.

En el sombreador de vértice, también convertimos la matriz de vista a 3x3 y luego a 4x4. Este truco elimina la componente de traslación, conservando la rotación (los valores de traslación están en la cuarta columna de la matriz de transformación). Esto fija el mapa cúbico en la posición de la cámara, permitiendo a la cámara virtual "observar" el entorno. La salida del Programa 9.2 es la misma que la del Programa 9.1.

Programa 9.2: Mapa cúbico OpenGL para cielo

Revisar el código adjunto

Mapeo ambiental

Al estudiar la iluminación y los materiales, consideramos el brillo de los objetos. Sin embargo, nunca modelamos objetos muy brillantes, como un espejo o algo de cromo. Estos objetos no solo tienen reflejos especulares; reflejan el entorno. Al observarlos, vemos elementos de la habitación, o incluso nuestra propia imagen reflejada. El modelo de iluminación ADS no permite simular este efecto.

Los mapas de textura cúbicos ofrecen una forma relativamente sencilla de simular superficies reflectantes (al menos parcialmente). La clave es *usar el mapa cúbico para texturizar el objeto reflectante*³. Para que parezca realista, se deben encontrar las coordenadas de textura que correspondan a la parte del entorno que se refleja en el objeto desde nuestro punto de vista.

La figura 9 ilustra la estrategia que consiste en combinar el vector de visión con el vector normal para calcular un *vector de reflexión*, el cual se utiliza posteriormente para obtener un

³ Este mismo método también es aplicable cuando se utiliza una bóveda celeste en lugar de un mapa de cielo, mediante la aplicación de la textura de la bóveda celeste al objeto reflectante.

texel de la textura en cubo. De esta manera, el vector de reflexión permite acceder directamente a la textura en cubo. Cuando la textura en cubo cumple esta función, se denomina *mapa de entorno* (*environment map*).

Ya calculamos los vectores de reflexión al estudiar el modelo de iluminación de Blinn-Phong. El concepto aquí es similar, pero ahora usamos el vector de reflexión para obtener un valor de un mapa de textura.

Esta técnica se denomina **mapeo ambiental o mapeo de reflexión**. Si el mapa de cubo se implementa mediante el segundo método descrito (en la sección anterior; es decir, como `GL_TEXTURE_CUBE_MAP` de OpenGL), OpenGL puede realizar la búsqueda en el mapa de textura de la misma manera que para texturizar el mapa de cubo.

Usamos el vector de visión y la normal de la superficie para calcular la reflexión del vector de visión en la superficie del objeto. Este vector de reflexión se usa para obtener la muestra directamente del mapa de textura.

La función `samplerCube` de OpenGL facilita esta búsqueda; recordemos que `samplerCube` se indexa mediante un vector de dirección de visión. Por lo tanto, el vector de reflexión es ideal para obtener el texel deseado.

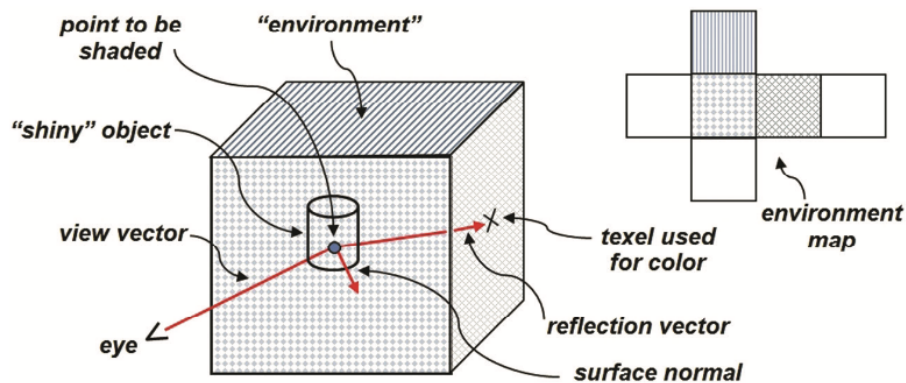


Figura 9: Descripción general del mapeo ambiental.

La implementación requiere muy poco código adicional. El programa 9.3 muestra los cambios en las funciones `display()` e `init()` y los shaders necesarios para renderizar un torus reflectante mediante mapeo ambiental. Los cambios están resaltados.

Cabe destacar que, si ya se utiliza la iluminación de Blinn-Phong, muchas de estas adiciones probablemente ya estén presentes. La única sección de código realmente nueva está en el shader de fragmento (en el método `main()`).

De hecho, podría parecer que el código resaltado en el programa 9.3 (secciones en amarillo) no es nuevo. Ya vimos código casi idéntico al estudiar la iluminación. Sin embargo, en este caso, los vectores normal y de reflexión se usan para un propósito completamente diferente.

Antes se usaban para implementar el modelo de iluminación ADS. Aquí se usan para calcular las coordenadas de textura para el mapeo ambiental. Resaltamos estas líneas para que sea más fácil seguir el uso de las normales y los cálculos de reflexión para este nuevo propósito. El resultado, que muestra un torus con efecto cromado mediante el mapeo ambiental, se muestra en la figura 10.



Figura 10: Ejemplo de mapeo ambiental para crear un toroide con efecto reflectante.

Programa 9.3: Mapeo de entorno

```
void display(GLFWwindow* window, double currentTime) {
    // the code for drawing the cube map is unchanged.
    ...
    // the changes are all in drawing the torus:
    glUseProgram(renderingProgram);

    // uniform locations for matrix transforms, including the transform for normals
    mvLoc = glGetUniformLocation(renderingProgram, "mv_matrix");
    pLoc = glGetUniformLocation(renderingProgram, "p_matrix");
    nLoc = glGetUniformLocation(renderingProgram, "norm_matrix");

    // build the MODEL matrix, as before
    mMat = glm::translate(glm::mat4(1.0f), glm::vec3(torLocX, torLocY, torLocZ));

    // build the MODEL-VIEW matrix, as before
    mvMat = vMat * mMat;
    invTrMat = glm::transpose(glm::inverse(mvMat));
```

```

// the normals transform is now included in the uniforms:
glUniformMatrix4fv(mvLoc, 1, GL_FALSE, glm::value_ptr(mvMat));
glUniformMatrix4fv(pLoc, 1, GL_FALSE, glm::value_ptr(pMat));
glUniformMatrix4fv(nLoc, 1, GL_FALSE, glm::value_ptr(invTrMat));

// activate the torus vertices buffer, as before
glBindBuffer(GL_ARRAY_BUFFER, vbo[1]);
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 0, 0);
glEnableVertexAttribArray(0);

// we need to activate the torus normals buffer:
glBindBuffer(GL_ARRAY_BUFFER, vbo[2]);
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 0, 0);
glEnableVertexAttribArray(1);

// the torus texture is now the cube map
glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_CUBE_MAP, skyboxTexture);

// drawing the torus is otherwise unchanged
glClear(GL_DEPTH_BUFFER_BIT);
glEnable(GL_CULL_FACE);
glFrontFace(GL_CCW);
glDepthFunc(GL_LEQUAL);

glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, vbo[3]);
glDrawElements(GL_TRIANGLES, numTorusIndices, GL_UNSIGNED_INT, 0);
}

```

Vertex shader

```

#version 430
layout (location = 0) in vec3 position;
layout (location = 1) in vec3 normal;
out vec3 varyingNormal;
out vec3 varyingVertPos;
uniform mat4 mv_matrix;
uniform mat4 p_matrix;
uniform mat4 norm_matrix;
layout (binding = 0) uniform samplerCube tex_map;

void main(void)
{
    varyingVertPos = (mv_matrix * vec4(position,1.0)).xyz;
    varyingNormal = (norm_matrix * vec4(normal,1.0)).xyz;
    gl_Position = p_matrix * mv_matrix * vec4(position,1.0);
}

```

Fragment shader

```

#version 430
in vec3 varyingNormal;
in vec3 varyingVertPos;
out vec4 fragColor;
uniform mat4 mv_matrix;
uniform mat4 p_matrix;
uniform mat4 norm_matrix;
layout (binding = 0) uniform samplerCube tex_map;

void main(void)
{
    vec3 r = -reflect(normalize(-varyingVertPos), normalize(varyingNormal));
    fragColor = texture(tex_map, r);
}

```

Si bien esta escena requiere dos conjuntos de shaders (uno para el mapa de cubo y otro para el torus), el Programa 9.3 solo muestra los shaders utilizados para dibujar el torus. Esto se debe a que los shaders para renderizar el mapa de cubo son idénticos a los del Programa 9.2. Las modificaciones realizadas en el Programa 9.2, que dieron como resultado el Programa 9.3, se resumen a continuación:

En **init()**:

- Se crea un búfer de normales para el torus (en realidad, esto se realiza en `setupVertices()`, que es llamado por `init()`).
- Ya no se necesita el búfer de coordenadas de textura para el torus.

En **display()**:

- Se crea la matriz de transformación de normales (denominada “norm_matrix” en el documento 7) y se vincula a la variable uniforme correspondiente.
- Se activa el búfer de normales del torus.
- El mapa de cubo se activa como textura del torus (en lugar de la textura “brick”).

En el shader de vértice (**vertex shader**):

- Se agregan los vectores normales y la matriz `norm_matrix`.
- Se envían los vértices y vectores normales transformados para calcular el vector de reflexión, similar a lo que se hizo para la iluminación y las sombras.

En el shader de fragmento (**fragment shader**):

- El vector de reflexión se calcula de forma similar a como se hizo para la iluminación.
- El color de salida se obtiene de la textura (ahora el mapa de cubo), utilizando el vector de reflexión como coordenada de textura.

El resultado, mostrado en la figura 10, es un excelente ejemplo de cómo un simple truco puede crear una ilusión convincente. Simplemente aplicando el mapa de entorno a un objeto, logramos que parezca ‘metálico’, sin haber implementado ningún modelo de material (ADS). Además, da la apariencia de que la luz se refleja en el objeto, aunque no se haya aplicado ninguna iluminación. En este ejemplo, incluso parece haber un reflejo especular en la parte inferior izquierda del torus, ya que el mapa de cubo incluye la reflexión de la luz solar en el agua.

Notas adicionales

Al igual que en el documento 5, cuando estudiamos las texturas, si bien SOIL2 facilita la creación y el mapeo de texturas en un mapa de cubo, puede impedir que el usuario aprenda algunos detalles importantes de OpenGL. Por supuesto, es posible crear e importar un mapa de cubo OpenGL sin SOIL2. Sin embargo, se recomienda usar una biblioteca para el manejo de imágenes, como stb_image.h⁴. Los pasos básicos se detallan en J. de Vries⁵ y se resumen así:

1. Copia el archivo de cabecera stb_image.h en el directorio de tu proyecto.
2. Usa glGenTextures() para crear una textura para el mapa de cubo y su referencia numérica.
3. Llama a glBindTexture(), especificando el ID de la textura y GL_TEXTURE_CUBE_MAP.
4. Lee los seis archivos de imagen con stbi_load().
5. Usa glTexImage2D() para asignar las imágenes a las caras del cubo.

El archivo de cabecera stb_image.h está incluido en SOIL2 o se puede instalar por separado. Para más detalles, consulta la página de J. de Vries.

Una limitación importante del mapeo ambiental, como se presenta en este documento, es que solo permite crear objetos que reflejen el mapa de cubo. Otros objetos renderizados en la escena no se reflejan en el objeto con mapeo de reflexión. Dependiendo de la escena, esto puede ser aceptable o no. Si hay otros objetos que deben reflejarse en un espejo o superficie cromada, se deben usar otros métodos. Un enfoque común utiliza el búfer de plantilla

⁴ <https://github.com/nothings/stb>

⁵ J. de Vries, “Learn OpenGL - Cubemaps”, <https://learnopengl.com/Advanced-OpenGL/Cubemaps>.

(mencionado en el documento 8) y se describe en varios tutoriales web⁶, pero está fuera del alcance de este texto.

No incluimos una implementación de bóvedas celestes, aunque son, en cierto modo, más simples que los skyboxes y menos propensas a distorsiones. Incluso el mapeo ambiental es más simple (al menos matemáticamente), pero el soporte de OpenGL para mapas de cubo suele hacer que los skyboxes sean más prácticos.

De los temas que se tratan en las últimas secciones de este curso, los skyboxes y las bóvedas celestes son, probablemente, los más sencillos conceptualmente. Sin embargo, lograr un resultado convincente puede llevar mucho tiempo. Hemos abordado brevemente algunos problemas que pueden surgir (como las líneas de unión), pero, dependiendo de los archivos de imagen de textura, pueden surgir otros problemas que requieren corrección. Esto es especialmente cierto cuando la escena está animada o cuando la cámara se mueve de forma interactiva.

También pasamos por alto el tema de la generación de mapas de textura cúbicos útiles y realistas. Existen herramientas excelentes para ello, siendo Terragen una de las más populares.

⁶ A. Overvoorde, “Depth and stencils”, <https://open.gl/depthstencils>